四川大学期末考试试题（闭卷）

（2022~2023 学年第 2 学期）　　A 卷

I. Single Selection　(2 points*5 = 10points)

B B C A B

II. Query Expression　（4 points×10=40 points）

Please use relational algebra language to express query requirements (1)- (4)：

(1) Find all the bank depositing account_numbers of the customer whose name is '张三'

$\Pi_{account\_number}(\sigma_{customer\_name='张三'}(customer \bowtie account))$

Or

$\Pi_{account\_number}(\sigma_{customer\_name='张三' \text{ and } customere.ID=account.ID}(customer \times account))$

(2) Find the customer IDs and names who has not borrow any loan from bank.

$\Pi_{ID, customer\_name}(customer) - \Pi_{ID, customer\_name}(\sigma_{customer.ID=loan.ID}(customer \times loan))$

Or

$\Pi_{ID, customer\_name}(customer) - \Pi_{ID, customer\_name}(customer \bowtie loan)$

(3) List ID and the summary of all account balances of every customer.

$\Pi_{ID, s1}(ID\ \mathcal{G}_{sum(balance)\ as\ s1}(\sigma_{account.ID=customer.ID}(account \times customer)))$

Or　$ID\ \gamma_{sum(balance)\ as\ s1}(\sigma_{account.ID=customer.ID}(account \times customer))$

Or　$ID\ \gamma_{sum(balance)\ as\ s1}(\sigma_{account.ID=customer.ID}(account))$

(4) List IDs of the customers who have borrowed loans from all branches.

$\Pi_{ID,branch\_name}(loan) \div \Pi_{branch\_name}(branch)$

or

$\Pi_{ID,branch\_name}(loan \bowtie branch) \div \Pi_{branch\_name}(branch)$

or

$\Pi_{ID,branch\_name}(loan \bowtie customer) \div \Pi_{branch\_name}(branch)$

Please useSQL language to express query requirements (5)-(10):

(5) Query the account_numbers of customers whose names include "君";

```
SELECT account_number
  FROM loan natural join customer
  WHERE customer_name like '%君%';
```

(6) Query the IDs of the customers who have at least one loan but no any account.

```
SELECT distinct ID
FROM  loan
WHERE  ID  not in
            ( SELECT ID
```

```
                    FROM   account);
```

Or

```
SELECT distinct ID
FROM   loan
except
select distinct ID
FROM account;
```

Or

```
SELECT ID
FROM loan
Group by loan.ID
HAVING count(loan_number)>=1
Except
Select distinct ID
From account;
```

(7) List each bank branch_names and the quantity of customers who have loans from the branch, and output the tuples in descending order of the quantity;

```
SELECT   branch_name, count(distinct ID) as quantity
FROM   loan
GROUP BY branch_name
ORDER BY quantity desc;
```

(8) Query IDs of the customers whose balance summary of his/her depositing accounts is greater than 10000；

```
SELECT   ID
FROM   account
GROUP BY ID
HAVING sum(balance) > 10000;
```

(9) Query IDs of the customers whose balance summary of his/her depositing accounts is the largest；

```
SELECT   ID
FROM   account
GROUP BY ID
HAVING sum(balance)>=all (SELECT sum(balance)
                          FROM   account
                          GROUP BY ID);
```

(10) Find the customer IDs who have borrowed loans from all the bank branches which the customer (ID= "A101") has borrowed loans from；

方法 1： Let X: all the bank branches which customer ID= "A101" has borrowed loans from
Let Y: all the bank branches which the specular customer who has borrowed loans from
Then : SELECT ID FROM loan WHERE not exist(X-Y);

```
SELECT ID
FROM loan A
WHERE not exists (
        (select distinct branch_name
         from loan
        where ID ='A101')
      except
       (select distinct branch_name
       from loan
```

```
        where ID =A.ID)   );
```

方法 2：选出某客户的条件是：　不存在某支行，客户 A101 贷款了，而该客户没有贷款记录。

```
Select ID
From customer
Where not exists
        (select * from loan L1
         Where ID='A101' and not exists
                        (select * from loan L2
                         Where L2.branch_name=L1.branch_name
                         And L2.ID=customer.ID)
```

III. Normalization　(10 points *2 =20 points)

1.

(1) Find the candidate keys of R.　If you think R has no candidate keys, write "no"　(3points)

A+=ABCDE=R, B+=BCDE, C+=CD, E+=E, 其它左部不含 A 的属性组合都不是码。

所以，候选码只有一个：A

(2) Is R in BCNF ? Please give the reason for your judgment.　(3 points)

不满足 BCNF.

如 B→C，非平凡，左部不含码，违反 BCNF 条件。

(3) Is F a canonical cover?　if no, please find a canonical cover for F.　(4 points)

不是。

Fc={ A →B, B→CE, C→D }

2.

(1) According to the above constraints, please write out　the reasonable functional dependencies holding on relation StudentProject（3 points）

F＝{sno→sname, sno→deptname, pno→pname, pno→pfund, (sno, pno)→reward }

(2) List all the candidate keys of StudentProject　（2 points）

候选码：(sno,pno)

(3) is StudentProject in 3NF? If it is not in 3NF, please decompse it into a set of 3NF relations and explain why your decomposition is lossless and dependency perseving（5 points）

不满足 3NF，如 sno→sname 违反了 3NF 条件；

用合成法进行分解：

Fc={ sno→sname, deptname, pno→pname,pfund, (sno, pno)→reward }

R1: student(sno,sname,deptname),　F1={ sno→sname,deptname }

R2: project(pno,pname,pfund),　F2={ pno→pname,pfund }

R3: participate(sno, pno,reward),　F3={(sno, pno)→reward }

因为 R1 ∩ R3→R1，R2 ∩ R3→R2 所以分解无损

因为 F1∪F2∪F3=F，所以分解依赖保持

IV. Concurrently Control and Recovery (10 points * 1=10 points)

(1) Is S1 a conflict serializable schedule? Please give the reason for your judgment。If so, S1 conflict equivalent to which serial schedule？（3 points）
S1 是冲突可串行化调度，T1 的 read(B)指令与 T2 的 read(B)可交换顺序，冲突等价于串行调度 T1,T2。

(2) Determine whether schedule S1 is a recoverable schedule, is a cascadeless schedule, and give the reasons.（2 points）
S1 是可恢复调度，当 T2 读了 T1 所写的数据 A，有 T2 的 commit 在 T1 的 commit 之后。
S1 是无级联调度，当 T2 读了 T1 所写的数据 A，有 T2 的 read(A)在 T1 的 commit 之后。

(3) Please add lock and unlock instructions to T1 and T2 to meet the two-phase locking protocol (2PL).（3 points）
T1:lock-X(A) read(A) writ(A) lock-S(B) read(B) commit unlock(A) unlock(B)
T2: lock-S(B) read(B) lock-S(A) read(A) commit unlock(A) unlock(B)
加锁后，保证了可得到可串行化调度。

(4) Whether T1 and T2 may produce deadlock after adding the two-phase locking protocol(2PL)? Please give the reason for your judgement.（2 points）
加锁后，不会产生死锁，因为 T1 只对数据项 A 申请排他锁，没有对数据 B 申请排他锁，T2 只对数据项 A，数据项 B 申请读锁， 不会造成互相等待的情况。

V. Database Design (10 points*2=20 points)

1.

Customer(customer_id , name, address);
Car (license_no, model, customer_id); //Fks: customer_id ->customer(customer_id)
Policy(policy_id);
Premium_payment (policy_id , payment_no, due_date, amount, received_on); //Fks: policy_id ->policy(policy_id)
Accident(report_id, date, place) ;
Participated (license_no, report_id); //Fks: policy_id ->policy(policy_id), report_id ->accident(report_id)
Covers(license_no,policy_id) // Fks: license_no ->car(license_no), policy_id ->policy(policy_id)

2.

.